

How High and How Fast: Edge-Computed Flood Stage and Trend-Derived Rate of Rise for River Catchment Early Warning

Kavya Vegunta

Student ID: X25156322

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25156322@student.ncirl.ie

Abstract—A rising river poses two separate questions to anyone responsible for the reach below it: how high is the water now, and how fast is it climbing. The first is a level to be checked against fixed flood-stage bands; the second is a rate, and a rate cannot be read from a single instant because it is a change measured across time. This paper presents a river-flood early-warning monitor built as a two-tier fog-to-cloud pipeline that answers each question on the tier suited to it. Five hydrological signals across two river reaches are produced by ten sensor processes; an edge relay folds each reach-and-signal stream into fixed ten-second windows, reduces each window in a single pass, and raises the flood stage locally from the window's peak, so only finished summaries and their alarms cross the network. A serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB stores the summaries, and a dashboard derives the rate of rise on read from the level trend across recent windows, a leading indicator that can fire before a fixed stage line is reached. The system was deployed to a real Amazon Web Services account; two faults that appear only on the managed runtime were found and fixed. An automated suite of 60 tests passes, and live verification confirmed both reaches streaming, alarms firing, and every pipeline indicator healthy.

Keywords—flood early warning, river monitoring, hydrology, rate of rise, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

A flooding river gives warning in two different currencies. One is its level: the stage, in metres, that can be compared against the advisory, watch, and warning marks a catchment authority has fixed for the reach. The other is its rate of rise: how quickly that level is climbing. The two are not interchangeable, and the difference is the whole of the early-warning problem. A river standing high but steady after a passed storm is a known, bounded hazard; a river still well under its advisory mark but rising a metre an hour is a different and more urgent one, because the level alone has not yet said what the slope already implies. Flash-flood forecasting turns precisely on buying lead time from the second signal before the first has crossed a line [1], [2], and instrumenting a catchment to read both continuously is now inexpensive with networked sensors [3], [4]. But a stream of raw readings is

not an answer. The person watching a reach does not want five live traces on five scales; they want a fused verdict: how high is each reach, and is it rising fast enough to matter.

The distributed-systems question is where those two verdicts are formed, and they do not both belong in the same place. Sending every raw sample from every gauge to a distant region to be judged there is wasteful in two directions at once: it spends bandwidth continuously on readings that individually seldom move a decision, and it defers the one moment that matters, the crossing of a stage band, until after a round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to summarise, filter, and evaluate rules before anything travels [5], [6], one expression of the broader movement of computation back toward the edge [7], [8]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that is idle most of the time [9]. The design in this paper puts the level question, which one window can answer, at the edge, and the rate question, which no single window can answer, in the tier that already holds the history a rate needs.

This project builds that two-tier design for a scaled but faithful slice of a catchment: two river reaches, designated reach-a and reach-b, each carrying the same five hydrological signals, run as ten independent sensor processes. An edge relay receives their readings, groups each reach-and-signal pair into a fixed window, reduces the window to summary statistics, raises the flood stage from the window's peak, and forwards only the finished window and any alarm it trips. A serverless cloud pipeline persists each window, and a dashboard turns the persisted history into a per-reach reading whose rate of rise is computed on read from the trend of recent windows.

The work pursues four objectives, each shown on a running system rather than only described. The first is a sensor and edge layer that genuinely windows and screens five heterogeneous signals at configurable rates. The second is a backend that scales through managed, elastic primitives rather than a single always-on server. The third is a serving layer that answers the level question from stored alarms and the

rate question from the stored trend, so a leading indicator is derived rather than stored. The fourth, and the one that separates a working system from a merely plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held deliberately to these layers at a two-reach scale. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the split between the two questions and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence from the running system. Section IV closes with a critical assessment of the trade-offs, limits, and future work.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the edge relay sharing one host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Edge Layer

Each of the ten sensor processes stands for one hydrological signal on one reach. It runs as a small asynchronous program whose sampler and dispatcher share a plain list on one event loop, so no lock is needed between producing a reading and shipping it. On every sampling tick the process advances a two-regime gauge: most of the time it drifts gently back toward a baseline with a little noise, but with a small probability it enters a short storm regime in which the value climbs in surging steps for a spell before settling again, and the result is clamped into a physically plausible band. That regime switch is what makes the traces behave like a catchment responding to a passing rain cell rather than like independent noise, and it is what lets a window's peak, its average, and the dashboard's trend line resemble real telemetry. The plausible bands differ sharply by signal, and that heterogeneity is what the design must carry: river level sits in metres, rainfall intensity in tens of millimetres per hour, flow velocity in metres per second, soil moisture as a percentage, and turbidity across hundreds of nephelometric units.

Two independent, separately configurable intervals drive each process: one sets how often a fresh value is sampled, defaulting to two seconds, and the other how often the accumulated readings are dispatched, defaulting to ten seconds. Keeping them genuinely separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact object naming the signal, the reach, and the unit, and carrying the batch of timestamped values gathered since the last dispatch, posted over HTTP to the relay's ingest endpoint. When a dispatch fails because the relay is briefly unreachable, the process puts the unsent batch back at the front of its buffer and retries on the next tick, so a transient outage delays readings rather than dropping them.

The edge relay is an asynchronous service that runs as its own container on the riverside host, distinct from the sensor

processes and from the cloud functions, and it does real work before anything leaves the edge. Its internal shape is deliberate. An arriving dispatch is validated and its readings are appended to an in-memory buffer keyed by the reach-and-signal pair; because the relay runs its handlers on a single event loop and one background task alone retires the buffers, the aggregation needs no lock, and the concurrent HTTP arrivals are serialised into one ordered stream that is simpler to reason about and to test than a lock-guarded structure shared by many threads. Once per window period a flush task drains the buffers, and each non-empty group is reduced in that single pass to a summary carrying the count, minimum, maximum, average, and latest value.

That summary, not any raw reading, is then screened against the reach's alarm rules. River level is treated as a multi-band flood stage read from the window's maximum: a peak at or above 3.5 m raises a flood advisory, at or above 4.5 m a flood watch, and at or above 5.5 m a flood warning, the highest matching band winning. The other signals carry one rule each on the statistic that suits them: an average rainfall above 40 mm/h raises a torrential-rain alarm, a peak flow velocity above 4.0 m/s a dangerous-current alarm, and an average soil moisture above 90% a saturated-catchment alarm, while turbidity is carried as context with no rule of its own because it colours the picture without being a hazard threshold. Reading the stage from the window's peak rather than its average is chosen on purpose: a flood mark is a level the water actually reached, so the maximum is the honest statistic, whereas rainfall and soil moisture are screened on the average because a single splashy sample should not raise an alarm a sustained condition has not. The limits live in two deliberately separate places, a set of screening functions that decide whether a summary trips and a declarative catalogue of the same numbers the dashboard can read back for display, so the thresholds can be disclosed without the disclosure ever driving the evaluation.

The relay's output is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls of up to the service's ten-per-batch limit, so the number of send operations stays proportional to how many signals close rather than to how fast they are sampled. The buffers are cleared as the summaries are drained, before the send is attempted, and the send is wrapped so a failure is logged and the cycle continues rather than stalling the edge; because the window has already been retired, a failed send drops that one window rather than wedging the relay, a bias toward keeping the edge live and current that suits a monitoring workload where the next window is seconds away. At start-up the relay resolves the queue's address once, retrying with backoff, because on a fresh deployment the queue may not yet exist when the host boots.

B. The Standout Decision: Two Questions on Two Axes of Time

The design's central choice is where each of the river's two questions is answered, and the answer follows from the

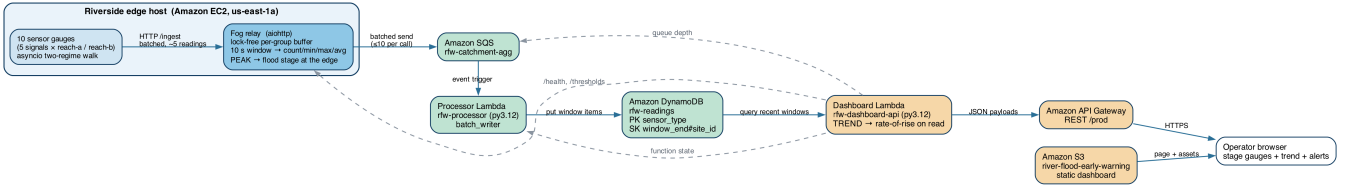


Fig. 1. Deployment topology. Ten sensor gauges and the aiohttp edge relay share one Amazon EC2 host at the riverside. The relay raises the flood stage from each window’s peak before anything travels (blue). Solid arrows are the ingest path, from the relay through Amazon SQS and the processor function into Amazon DynamoDB, and the serving path, from the dashboard function’s read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3; the dashboard derives the rate of rise from the stored trend on read (amber). The dashed edges are the three further status sources the dashboard polls for its health strip: the relay’s health endpoint, the queue’s depth, and the processor function’s state.

mathematics of the questions themselves. How high the water stands is a property of a single instant, so a single window can answer it: the relay reads the flood stage from the window’s peak and attaches it at the edge, where it costs nothing to keep and where an alarm can be raised the moment the water reaches a mark, without waiting for the cloud. How fast the water is rising is not a property of any instant. A rate is a difference in level divided by the time between two levels, so it cannot be computed inside one ten-second window at all; it needs a sequence of windows spread across real elapsed time. Making the edge answer it would mean making the edge stateful, holding a rolling history of past windows against the current one, which would trade away exactly the lock-free, memory-light simplicity that lets the relay keep up.

So the rate of rise is derived one tier later, on read, where the history already lives. The dashboard queries the recent level windows for a reach, takes the earliest and latest that carry an average and a closing timestamp, and divides the change in average level by the real elapsed time between their timestamps to yield a rate in metres per hour, smoothed across the span rather than taken from one noisy window. A rate at or above 8 m/h flags a rapid-rise condition, and a smaller band classifies the reach as rising, steady, or falling for the operator’s eye. The point of computing it here is not convenience but correctness of place: the rate of rise is a leading indicator, and a river can sit below its advisory stage while rising fast enough to breach within the hour, so the value most worth watching is precisely the one no single edge window can produce. Splitting the instantaneous question from the trend question, and putting each on the tier whose cost model fits it, is what lets the edge stay stateless and cheap while the leading indicator is still available where the stored windows make it free to compute.

C. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface: a queue receives the relay’s summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path lets ingestion and querying scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The relay could invoke the storage function directly and synchronously, saving a hop, but that binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the relay stalls as the back-pressure propagates toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the relay treat a summary as delivered the moment it is enqueued [10]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are what an intermittent riverside uplink needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions, which fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts [9]. The recognised cost is the cold start after idleness [11]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard’s own polling keeps the interface function warm.

Key-value store instead of a relational database. The access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the overhead of a relational server, the balance the original Dynamo work set out and the managed service now carries [12], [13]. The key design follows straight from the query. Each stored window is keyed on its signal type as the partition and on a sort value that leads with the window’s closing timestamp and then the reach identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan. Because the reach is only the tail of the sort value and not part of the partition key, the two reaches of one signal share a partition and are separated in the reader, and the same query that feeds the level trend for a reach is the query the rate of rise is computed from.

D. Serving Layer and Security Posture

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint, and object

storage holds the static frontend. The read function is not a second copy of the serving logic: the same view functions that the local server calls are the ones a small gateway adapter dispatches to, so each request path is handled by one shared function whether the code runs as a local server or as a deployed function, and the routing that only the deployed function exercises is the part most exposed to a deployment-only mistake. The surface is small and read-only: a reaches endpoint giving each reach’s current stage, trend, level, rate of rise, and open alarms together with the worst stage across the catchment; a readings endpoint returning a recent series for one signal to feed the trend chart; a thresholds endpoint that forwards the relay’s published limits; a backend-statistics endpoint; and a health endpoint. The health response is assembled from four live sources at once, the store for the freshest window, the queue for its reachability, the processor function for its state, and the relay for its own reachability, and it returns those flags with a pipeline flag that is true only when a window actually reached the store within the last thirty seconds, so a reachable-but-stale pipeline is distinguished from a healthy one.

On the page, each reach is a card with a vertical stage gauge drawn against the three flood-stage bands, a status of normal, advisory, watch, or warning taken from the stored stage alarms, the current level and its rate of rise with a rising, steady, or falling label, and the reach’s other four signals with their values; a header reports the worst stage across the catchment, and an alarm strip names any firing condition per reach. Below the cards, a two-reach level trend is drawn against the stage bands so the quantity that defines a flood is the one shown over time, and a reference panel lists the thresholds the relay applies. A fixed browser-side timer re-fetches the reaches, health, and statistics every few seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it, substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the relay’s health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, so what they disclose is bounded to what the dashboard already renders, while the cloud tier relies on the managed services’ own access control, on the edge host admitting only the one port the relay needs with no remote login, and on the frontend bucket being public read-only.

III. IMPLEMENTATION

The sensor processes, the edge relay, and both cloud functions are written in Python and run on the Python 3.12 runtime. The relay serves its ingest, health, and threshold endpoints on an asynchronous web framework whose event loop also runs the flush task, while the sensor processes post their batches

with an asynchronous HTTP client. The dashboard reader is a standard-library HTTP server locally, and its request handlers and payload builders are reused unchanged inside the deployed function through a thin gateway adapter, so no route logic is duplicated between the two runtimes. The AWS SDK for Python carries the queue, table, and function calls. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 60 tests spread across every component: the sensor gauge’s sampling and dispatch, the relay’s ingest validation, its buffer drain, and its live HTTP endpoints, the window aggregation and its arithmetic, the flood-stage bands and the single-signal screeners at and around each limit, the batched publisher, the processor’s normalisation of a queued summary into a stored item and its composite sort value, the dashboard’s query shaping and health assembly, the read-time reach status, the rate-of-rise derivation, and the gateway adapter’s routing. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a call would issue rather than depending on a live service. The screening tests are pointed at the boundaries, checking a value one step inside a limit and one step outside it, so a band that should fire and one that should not are both pinned down. The rate-of-rise tests deserve particular note, because the derivation is the design’s distinctive part: they check that a rise is divided by real elapsed time between the windows’ closing timestamps, that a series of fewer than two points yields no rate rather than a divide-by-zero, and that a non-positive elapsed span is rejected, so the leading indicator is correct at its edges as well as in the ordinary case. The suite runs on every push through GitHub Actions, so a change that breaks a screening boundary, a query shape, the sort value, or the rate arithmetic fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue’s retry semantics were reasoned about from the service documentation [14] and then confirmed against the live deployment. Standing up a local emulator for every test was rejected as slower and still not the real service, and the gap it leaves is exactly what the live verification below is for.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated in its own workspace so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds

the queue, both functions with their triggers and permissions, the table, the gateway with its single catch-all route, and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the relay and the ten sensor containers under Docker Compose; the managed queue, the two functions [15], the table, the managed gateway [16], and the frontend bucket make up the cloud side, all carrying a common prefix so the deployment is easy to identify and remove as a unit, with a fixed public address in front of the edge host so the read function reaches the relay across restarts.

Two faults surfaced only once the system ran on the managed platform, and both are worth recording because neither can appear in a local run. The first was at the edge. The project had been developed and tested against a local emulator, and its container definition wired the relay, the sensors, and the emulator together for that purpose; it carried no separate definition for the real cloud, where the processor and the dashboard are functions rather than containers and the queue is a managed service. On the managed host the edge could not start, because the boot step looked for a cloud-specific container definition that did not exist. Adding one, running only the relay and the ten sensors against the real queue through the host's own role, let the edge come up. The second fault was in the browser and only over time. The level-trend chart sized its drawing surface from the width it had just rendered at and wrote that width back, scaled by the display's pixel ratio, on every refresh; on a high-density display each refresh therefore made the surface wider than the last, and after enough cycles it grew past what the browser would allocate and stopped drawing altogether. A single glance showed a correct chart, so only a real browser left running surfaced it; sizing the surface from its fixed container instead, and pinning it in the stylesheet, fixed it. Both are exactly the class of failure a fixture cannot reach, because a fixture imports the code on the same machine and never renders a page over many minutes.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed board during that check. Within a few minutes of start-up the pipeline reported healthy end to end: the relay, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both reaches rendered all five signals with their current value; their stage gauges filled against the flood bands; and during the check the catchment escalated to a warning state with flood-warning, torrential-rain, dangerous-current, and rapid-rise alarms firing across the reaches, the rapid-rise flag being the derived leading indicator rather than a stored one. The two-reach level trend filled and advanced against the stage bands, and the stored-record count climbed across successive polls rather than staying static, which confirms the ingestion function was draining the queue and writing on an ongoing basis. Reaching the board from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

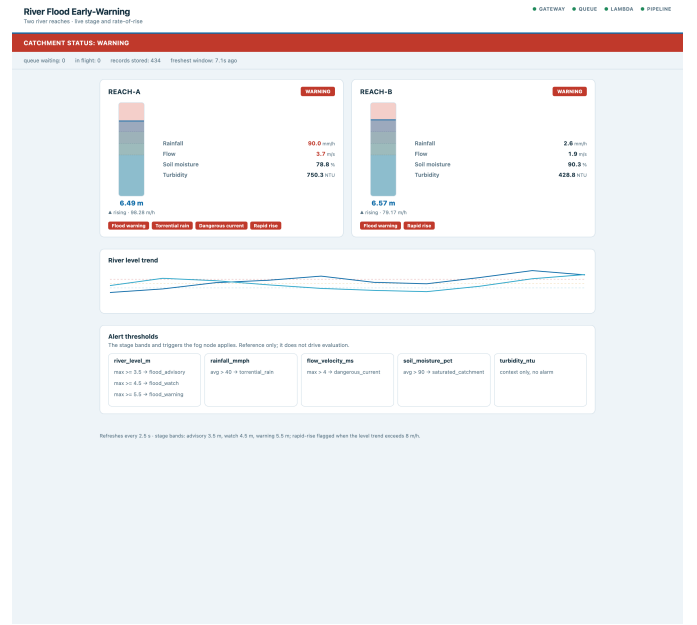


Fig. 2. The deployed board reading from the live endpoint. Each reach shows a stage gauge against the flood bands, its status, level and rate of rise, and its five signals; the header reports the worst catchment stage; the alarm chips name the firing conditions; the two-reach chart shows the level trend against the stage bands; and the reference panel lists the thresholds the relay applies.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five hydrological signals across two river reaches into a live answer to the two questions a rising river poses, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each reach-and-signal stream and raises the flood stage locally from the window's peak, so only compact summaries cross the network and a stage crossing is flagged in the window it occurs. The serving tier answers the second question where it can be answered cheaply, deriving the rate of rise on read from the stored level trend so that a leading indicator no single edge window could produce is still available without making the edge stateful. Between them the cloud tier scales through decoupling and elasticity rather than a pre-sized server. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 60 automated tests guarding its behaviour and continuous integration running them on every change.

Building this monitor left two lessons worth recording. The clearer one is that the tier a computation belongs on follows from the mathematics of the computation: the flood stage is a property of an instant and so belongs at the edge, while the rate of rise is a property of a span and so belongs where a span of history is already stored, and forcing either onto the wrong tier would cost either lead time or the edge's lock-free simplicity. The other is that deploying to a real account is a different claim from passing tests. The two faults that mattered here, an edge

that had only ever been wired for a local emulator and a chart that grew its own drawing surface until it broke, both passed every unit test and both failed only on the managed platform and in a real browser left running, because a test imports the code on the machine that wrote it and never renders a page over time. Each had to be reasoned about against real behaviour rather than assumed away, and finding them was the clearest argument in the project for verifying against the live system.

The work has clear limits. It runs at a two-reach scale with simulated gauges, its flood-stage and rate thresholds are fixed rather than learned or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a catchment of many reaches and real gauge feeds, to move the fixed thresholds to a managed configuration an operator could adjust without a redeploy, to extend the rate-of-rise derivation into a short forecast horizon that projects a breach time from the current slope, and to add authentication and a custom domain in front of the public endpoint. None of these changes the shape of the pipeline, which is the point: the design of answering the level question at the edge and the rate question from the stored trend carries from two reaches to many without rethinking the tiers.

The source code and deployment configuration are available at [GitHub repository URL].

REFERENCES

- [1] H. A. P. Hapuarachchi, Q. J. Wang, and T. C. Pagano, "A Review of Advances in Flash Flood Forecasting," *Hydrological Processes*, vol. 25, no. 18, pp. 2771–2784, 2011.
- [2] R. Krzysztofowicz, "The Case for Probabilistic Forecasting in Hydrology," *Journal of Hydrology*, vol. 249, no. 1–4, pp. 2–9, 2001.
- [3] E. Basha and D. Rus, "Design of Early Warning Flood Detection Systems for Developing Countries," in *Proc. Int. Conf. Information and Communication Technologies and Development (ICTD)*, 2007, pp. 1–10.
- [4] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [6] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [8] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [9] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [10] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [12] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [13] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [14] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [15] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [16] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>